AI Design System Starter Guide

From the AI Design System: The Know-How Masterclass

MOHAMED ISMAIL

GETTING STARTED

Setup

Prerequisites

Claude Code	Installed and authenticated
Figma account	With Edit access (Dev Mode recommended)
Figma MCP	Connected in Claude Code settings

Setup Steps

1. **Create your copy**
   Click "Use this template" on GitHub, or download the ZIP. Open it in your editor.

2. **Duplicate the Figma file**
   Open the session Figma file. Click "Duplicate to your drafts." Copy the file key from the URL — the string after `/design/`.

3. **Configure CLAUDE.md**
   Open `CLAUDE.md` and replace every `[YOUR_FIGMA_FILE_KEY]` with your file key.

4. **Verify the connection**
   Open Claude Code in this directory and ask:

```
# Your first prompt
What pages exist in my Figma file?

# Expected output:
Utility, Foundations, Screens, Button,
Jumbotron, Artwork, Screens with components
```

> **No build system.** There is no `package.json` or build step. Everything runs through `use_figma` via the Figma MCP.

WHAT YOU BUILT

The Five Artifacts

Everything you created during the session is included in this starter kit as a working reference.

#	ARTIFACT	LOCATION	WHAT IT IS
1	**Primitive tokens**	`foundations/`	Raw color ramps (8 hues x 50-900), 4px-grid spacing scale, and typography primitives (Inter)
2	**Semantic tokens**	`tokens/`	Purpose-based aliases with Light + Dark modes. Categories: background, text, icon, border
3	**Components**	`components/`	Button with 18 variants (3 styles x 3 sizes x icon toggle), fully bound to semantic tokens
4	**CLAUDE.md**	`CLAUDE.md`	8-section persistent context file — architecture, conventions, critical rules, discovered rules
5	**Documentation skill**	`.claude/skills/`	Reusable workflow that generates component documentation from live Figma data

```
# The three-tier architecture

Primitive  (color/yellow/500 = #FFD600)
   |
Semantic  (background/brand/solid -> primitive)
   |
Component (button/background/active -> semantic)
```

> **The discipline:** Components reference semantic tokens. Semantic tokens alias primitives. Primitives never get used directly.

REUSABLE WORKFLOW

Skill Reference

One Claude Code skill is included in `.claude/skills/`. It activates automatically when you use the trigger phrase.

component-doc-writer

Auto-generate component documentation from Figma

"Document the Button component" / "write docs for [component]"

Pulls variants, props, states, and token bindings from live Figma. Writes a structured markdown doc with anatomy, variant matrix, and token tables. Stubs judgment sections (When to use, Do/Don't, Teaching notes) with TODO markers for you to fill.

How it works

STEP	WHAT HAPPENS
1. Pull	Reads component metadata, variable definitions, and screenshot from Figma MCP
2. Fill	Maps live data to the doc template — variants, props, states, tokens, anatomy
3. Stub	Marks judgment sections as `TODO:` — never fabricates opinions
4. Write	Outputs `docs/components/<name>.md` with hero screenshot

Example workflow

```
# 1. Build a component
"Build an Input Field component in Figma.
 Variants: Size (sm/md/lg) x Type (default/outlined).
 Bind every fill/border/text to semantic tokens."

# 2. Document it
"Document the Input Field component"

# 3. Fill the TODOs yourself
# Open docs/components/input-field.md
# Search for TODO: and fill in judgment sections
```

> **Hard rule:** The skill never fabricates tokens or invents variant combinations. If Figma doesn't have it, the doc flags it as `(no token bound)`.

COPY & ADAPT

Prompts Quick Reference

Replace `[BRACKETED]` placeholders with your project specifics. Full versions in `docs/prompts.md`.

Pillar 1 — Tokens

CREATE PRIMITIVES

```
Create a `Primitive: Colors` collection from the palette at
[PALETTE_NODE_ID]. Group by hue. Naming: `color/{hue}/{shade}`.
```

CREATE SEMANTICS

```
Create a `Semantic: Colors` collection following the canvas at
[CANVAS_NODE_ID]. Alias every value to Primitive: Colors. Naming:
`{property}/{role}/{state}`.
```

ADD DARK MODE

```
Add a Dark mode to Semantic: Colors. Cover every Light-mode token. Explain
the pairing logic before writing.
```

Pillar 2 — Specs

BUILD A COMPONENT

```
Build a [COMPONENT] component in Figma. Variants: [matrix]. Bind every
fill/border/text/icon to a Semantic token. Hug-content sizing.
```

BOOTSTRAP CLAUDE.MD

```
/init
Then: "Project anchor for [SYSTEM] — 8-section structure: identity,
architecture, reference, conventions, critical rules, workflow patterns, file
references, discovered rules."
```

DOCUMENT A COMPONENT

```
Document the [COMPONENT_NAME] component.
```

When output drifts

REDIRECT

```
You bound [SURFACE] to `[TOKEN]`. We want [INTENT]. Re-examine — is the
token missing, or am I wrong about the intent?
```

Anti-patterns

BAD PROMPT	WHY IT FAILS
`Build me a button component`	No references, no constraints. Agent invents everything.
`Add dark mode`	No pairing rules, no scope. Agent invents conventions.
`We want a button with hover`	Narrative, no addressable refs. Agent guesses at tokens.

MENTAL MODEL

The Four Pillars

TOKENS → SPECS → AUDIT → AUTOMATION

PILLAR 1

Tokens

The values. Variables for color, type, spacing. Three tiers, each consuming the layer below.

PILLAR 2

Specs

The rules. CLAUDE.md, component docs, naming conventions. Persistent memory for every session.

PILLAR 3

Audit

The enforcement. Scripts that scan for drift — hardcoded values, missing bindings, naming violations.

PILLAR 4

Automation

The loop. Scheduled triggers that fire audits while you sleep. Output goes where the team sees it.

> **Each pillar earns the next.** Skip any one and the rest collapse. Tokens without specs are guesswork. Specs without audit are honor-system. Audit without automation runs only when someone remembers.

For each pillar, ask

PILLAR	DIAGNOSTIC QUESTION
Tokens	Could a new contributor produce a screen with zero raw hex / px / font names?
Specs	Does a fresh AI session know your architecture without you re-explaining it?
Audit	If someone bypassed your tokens tomorrow, how long would it take to notice?
Automation	Does drift surface to a human within 7 days, automatically?

If any answer is *no* or *I don't know*, that pillar isn't load-bearing yet. That's where you start.

PERSISTENT CONTEXT

CLAUDE.md Cheat Sheet

The 8-section structure. Fill it when you start a project. Update it the moment you discover a rule. A template is at `docs/CLAUDE.md.template`.

Section 1
Project Identity
What this is. Figma file key, default page, key node IDs. Two lines max.

Section 2
Architecture Summary
The tier model. Primitive → Semantic → Component. How it flows.

Section 3
Reference Tables
Collections, modes, variable counts, pages, component sets, scopes.

Section 4
Conventions
Variable naming patterns, component naming, file layout.

Section 5
Critical Rules
Start with 3. Grow to 30. Numbered, terse, each with a reason.

Section 6
Workflow Patterns
How the agent approaches work. Inspect first. Atomic steps. Return IDs.

Section 7
File References
Local repo tree, Figma node IDs, external skill references.

Section 8
Discovered Rules
The living log. Append the moment a rule surfaces. Don't tidy.

> **Section 8 is the most important.** Every time you hit a surprise — a font that won't load, a binding that renders wrong, a scope that pollutes the picker — append it immediately. Not at end of day. Immediately.

The most common mistake

Writing critical rules before you've discovered them in real work. Delete aspirational rules. Keep only what you've actually hit.

YOUR 90-DAY PLAN

Next Steps

Three phases. Lowest political cost first. Full version in `docs/next-steps.md`.

Phase 1 — CLAUDE.md, solo

`Week 1 · ~2 hours · Zero permission needed`

Save `CLAUDE.md.template` as `CLAUDE.md` in one project. Fill 5 of 8 sections. Use Claude Code for one real task. Add the first discovered rule. Prove to yourself that persistent context changes what the AI produces.

Phase 2 — Audit, still solo

`Weeks 2–4 · ~4–6 hours · Still just you`

Build an audit script for your system. Run it. Don't fix everything — just look at what surfaces. Pick one drift case and fix it properly. Add a critical rule to CLAUDE.md based on what you learned.

Phase 3 — Schedule & share

`Month 2+ · Now it's team-visible`

Schedule the audit weekly. Share the most surprising finding. Have one-on-ones with people who respond. Propose CI integration when 2–3 teammates are running their own CLAUDE.md.

> **Monday homework:** Don't try the whole stack. Pick one task from Phase 1. Open a project, create `CLAUDE.md`, fill in identity + architecture + 3 rules, and run one prompt. That's it.

Common failure modes

FAILURE	FIX
CLAUDE.md is aspirational, not battle-tested	Delete rules you haven't hit. Keep only discovered ones.
Audit flags too much, team ignores it	Tune rules until clean runs are common. Errors should be rare.
Team adopts but doesn't update	Add to PR template: "What did this PR teach us?"
You burn out before Phase 3	Phases 1-2 are solo. Don't bring others in early.

AI is the surface.
The architecture is yours.

Discipline first. Automation second.

Mohamed Ismail
desmismail@gmail.com